Chương 3
Index-Chỉ mục

GVHD: TS. Phan Thị Thể

FIT – HCM-UTE

Email: thept@hcmute.edu.vn

INDEX

1. Tổng quan về Lưu trữ và Chỉ mục
2. Chỉ mục dựa trên cấu trúc cây (Tree-Structured Indexing)
3. Chỉ mục dựa trên phương pháp băm (Hash-based indexing)
4. So sánh chi phí I/O của các tổ chức tệp và chỉ mục
5. Thiết kế vật lý và tinh chỉnh chỉ mục (index selection & tuning)
6. Bài tập

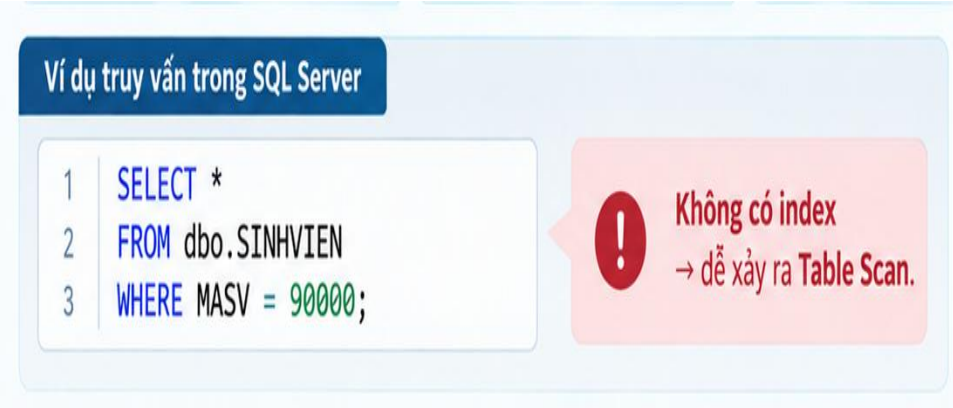
Tổng quan về Lưu trữ và Chỉ mục

Tổng quan về Lưu trữ và Chỉ mục

- Một hệ quản trị cơ sở dữ liệu (DBMS) lưu trữ một lượng dữ liệu khổng lồ và phải đảm bảo tính bền vững (persistence) qua các lần thực thi chương trình. Do đó, dữ liệu được lưu trữ trên các thiết bị bộ nhớ ngoài (chủ yếu là đĩa từ - magnetic disks) và chỉ được nạp vào bộ nhớ chính (RAM) khi cần xử lý .
- **Đơn vị truyền tải dữ liệu:**
  - Đơn vị thông tin được đọc hoặc ghi từ đĩa vào RAM là một **Trang (Page)** hoặc khối (Block). Kích thước trang điển hình là **4KB** hoặc **8KB**.
- **Mô hình chi phí I/O:**
  - Chi phí truy xuất đĩa (Page I/O) luôn là yếu tố thống trị hiệu năng của các thao tác dữ liệu cơ bản. Việc đọc/ghi các trang vật lý xếp liên tiếp nhau (sequential access) có chi phí rẻ hơn rất nhiều so với truy xuất ngẫu nhiên (random access) do giảm thiểu được thời gian dịch chuyển đầu từ (seek time) và thời gian quay của đĩa (rotational delay).
- **Mã định danh bản ghi (Record ID - RID):**
  - Mỗi bản ghi trong một tệp dữ liệu được định danh duy nhất bằng một **RID**. RID có cấu trúc vật lý dạng [Page ID, Slot Number], giúp hệ thống xác định chính xác địa chỉ đĩa của trang chứa bản ghi đó chỉ bằng một lần truy xuất.

Tổng quan về Lưu trữ và Chỉ mục

- Một **Tổ chức tệp (File Organization)** là phương pháp sắp xếp các bản ghi trên đĩa. Tổ chức tệp đơn giản nhất là tệp không thứ tự (**Heap file**), nơi dữ liệu được chèn một cách ngẫu nhiên.
- **Chỉ mục (Index)** là một cấu trúc dữ liệu bổ trợ được xây dựng trên đĩa nhằm tối ưu hóa việc tìm kiếm và truy xuất các bản ghi thỏa mãn một điều kiện lọc cụ thể.
  - **Khóa tìm kiếm (Search Key):** Là một thuộc tính hoặc một tổ hợp các thuộc tính (khóa phức hợp) của bảng dữ liệu dùng làm điều kiện để tìm kiếm trong chỉ mục.
  - *Lưu ý đặc biệt từ giáo trình:* Cần phân biệt rõ khái niệm **Khóa tìm kiếm (Search Key)** của chỉ mục và **Khóa chính (Primary Key)/Khóa ứng viên (Candidate Key)** của bảng dữ liệu. Khóa tìm kiếm không nhất thiết phải có giá trị duy nhất trong bảng.



RID (Row Identifier)

```
RID = [Page ID, Slot Number]
```

RID xác định vị trí chính xác của một bản ghi trong file dữ liệu (heap table).

Tổng quan về Lưu trữ và Chỉ mục

- Trong một tệp chỉ mục, các phần tử được lưu trữ được gọi là **thực thể dữ liệu chỉ mục (data entry)**, ký hiệu chung là k^* (với k là giá trị của khóa tìm kiếm). Có **3 phương án thiết kế vật lý** cho thực thể dữ liệu chỉ mục k^*.

Tổng quan về Lưu trữ và Chỉ mục

1 Alternative 1

k* = bản ghi dữ liệu thực tế

Trang dữ liệu (Data Page)

MASV	HOTEN	LOP	...
90000	Nguyễn A	KTPM	...
90001	Trần B	KTPM	...
90002	Lê C	HTTT	...
...	...	...	...

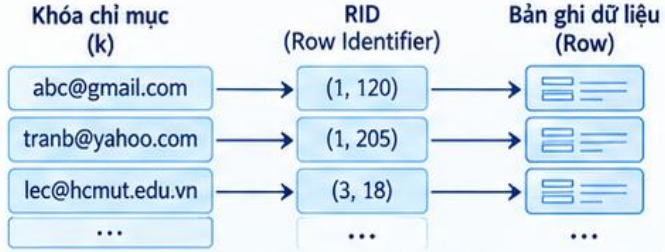
Ít truy xuất gián tiếp, nhưng mỗi bảng chỉ có tối đa 1 kiểu tổ chức này.

Ví dụ trên SQL Server

```
CREATE UNIQUE CLUSTERED INDEX
IX_SINHVIEN_MASV ON dbo.SINHVIEN(MASV);
```

2 Alternative 2

k* = <k, RID>



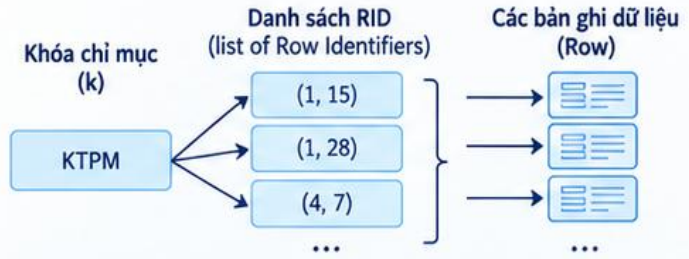
Phổ biến cho nonclustered index; gọn nhẹ, tạo được nhiều chỉ mục.

Ví dụ trên SQL Server

```
CREATE NONCLUSTERED INDEX
IX_SINHVIEN_EMAIL ON dbo.SINHVIEN(EMAIL);
```

3 Alternative 3

k* = <k, danh sách RID>



Tiết kiệm khi có nhiều giá trị trùng lặp; quản lý phức tạp hơn.

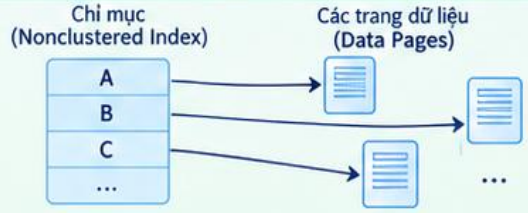
Clustered vs Nonclustered

Clustered (Chỉ mục cụm)



Các trang dữ liệu được sắp xếp theo khóa

Nonclustered (Chỉ mục không cụm)



Clustered: dữ liệu lưu gần theo thứ tự khóa. **Nonclustered:** index tách biệt, trỏ đến dòng dữ liệu.

Primary Index vs Secondary Index

Tiêu chí	Primary Index (Chỉ mục chính)	Secondary Index (Chỉ mục phụ)
Thuộc tính dùng làm khóa	MASV (khóa chính)	LOP hoặc EMAIL (không phải khóa chính)
Tính duy nhất	Thường là duy nhất (unique)	Có thể trùng lặp
Số lượng trên một bảng	Thường chỉ 1	Có thể có nhiều
Mục đích	Xác định duy nhất bản ghi	Hỗ trợ các truy vấn khác

Clustered index rất mạnh cho truy vấn khoảng (BETWEEN, >, <).

Primary index thường unique; Secondary index có thể trùng lặp.

Tổng quan về Lưu trữ và Chỉ mục

- Phương án 1 (Alternative 1):

  k^*=Bản ghi dữ liệu thực tế (với giá trị khóa tìm kiếm k)

- **Đặc điểm:**
  - Thực thể chỉ mục chính là bản ghi dữ liệu hoàn chỉnh. Chỉ mục sử dụng Phương án 1 đóng vai trò là chính tổ chức tệp dữ liệu đó (gọi là **Clustered File** hoặc **Index-Organized File**).
- **Ưu điểm:**
  - Tiết kiệm bộ nhớ và tránh được bước truy xuất gián tiếp qua RID.
- **Hạn chế:**
  - Một bảng dữ liệu chỉ có thể xây dựng tối đa **một** chỉ mục theo Phương án 1 để tránh nhân bản dữ liệu.

Tổng quan về Lưu trữ và Chỉ mục

- Phương án 2 (Alternative 2):

$$k^* = \langle k, rid \rangle$$

- **Đặc điểm:**
  - Thực thể chỉ mục là một cặp gồm giá trị khóa *k* và RID của bản ghi dữ liệu thực tế chứa khóa đó.
- **Ưu điểm:**
  - Kích thước thực thể chỉ mục nhỏ, có thể xây dựng nhiều chỉ mục độc lập trên cùng một bảng dữ liệu.

Tổng quan về Lưu trữ và Chỉ mục

- Phương án 3 (Alternative 3):

$$k^* = \langle k, [rid_1, rid_2, \ldots, rid_n] \rangle$$

- **Đặc điểm:**
  - Thực thể chỉ mục gồm giá trị khóa *k* và một danh sách các RID của tất cả các bản ghi dữ liệu có cùng giá trị khóa *k* đó.
- **So sánh với Phương án 2:**
  - Phương án 3 tối ưu hóa không gian lưu trữ tốt hơn do không phải lặp lại giá trị khóa *k* cho các bản ghi trùng lặp. Tuy nhiên, độ dài của mỗi thực thể chỉ mục là thay đổi (variable-length), làm tăng độ phức tạp khi quản lý trang chỉ mục.

Tổng quan về Lưu trữ và Chỉ mục

- Phân loại Chỉ mục theo các thuộc tính Vật lý
  - **Chỉ mục cụm (Clustered Index) và Chỉ mục không cụm (Unclustered Index)**
  - **Chỉ mục chính (Primary Index) và Chỉ mục phụ (Secondary Index)**

Tổng quan về Lưu trữ và Chỉ mục

Chỉ mục cụm (Clustered Index) và Chỉ mục không cụm (Unclustered Index)

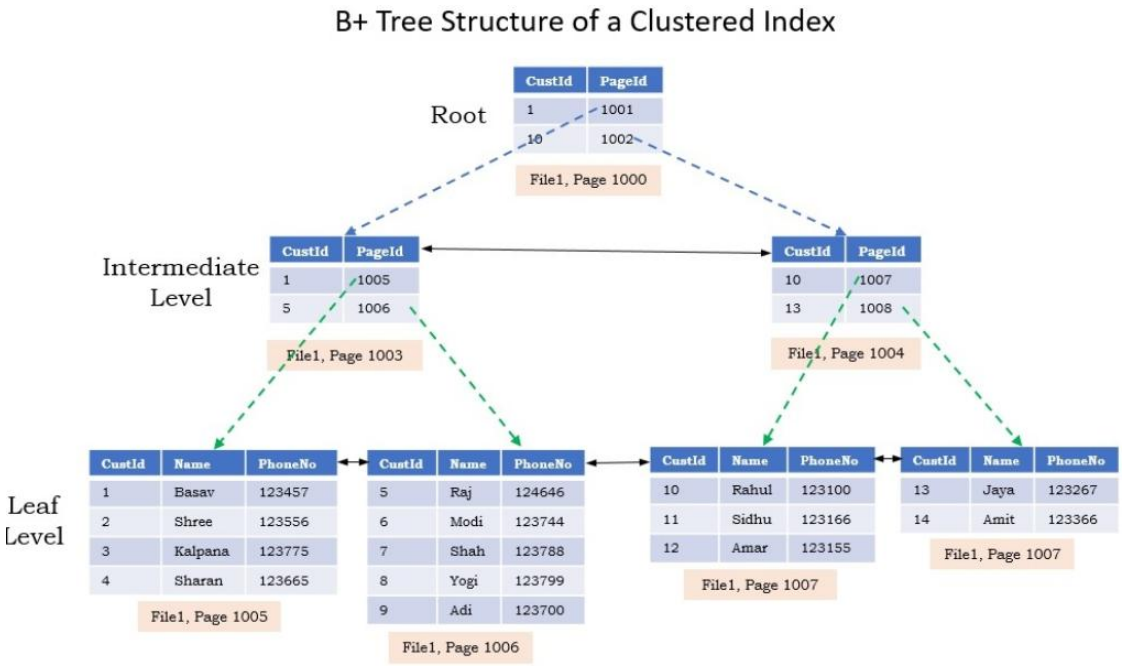
- **Chỉ mục cụm (Clustered Index):** Là chỉ mục có trật tự sắp xếp của các thực thể dữ liệu chỉ mục (k^*) trùng khớp (hoặc rất gần) với trật tự sắp xếp vật lý của các bản ghi dữ liệu thực tế trên đĩa.
  - *Hệ quả:*
    - Chỉ mục sử dụng **Phương án 1** luôn là chỉ mục cụm. Nếu chỉ mục sử dụng **Phương án 2 hoặc 3**, nó chỉ có thể là chỉ mục cụm nếu tệp dữ liệu gốc được sắp xếp theo đúng khóa tìm kiếm. Vì việc duy trì tệp dữ liệu được sắp xếp vật lý hoàn hảo khi có cập nhật là cực kỳ tốn kém, trong thực tế, chỉ mục cụm thường đồng nghĩa với việc tổ chức tệp theo Phương án 1.
  - *Chi phí I/O tìm kiếm khoảng (Range Search):*
    - Thể hiện sự vượt trội tuyệt đối của chỉ mục cụm. Khi tìm các bản ghi trong một khoảng giá trị, chỉ mục cụm chỉ cần truy xuất một vài trang liên tiếp. Ngược lại, chỉ mục không cụm có thể phải thực hiện mỗi lần I/O ngẫu nhiên cho **mỗi** bản ghi thỏa mãn, gây suy giảm hiệu năng nghiêm trọng.

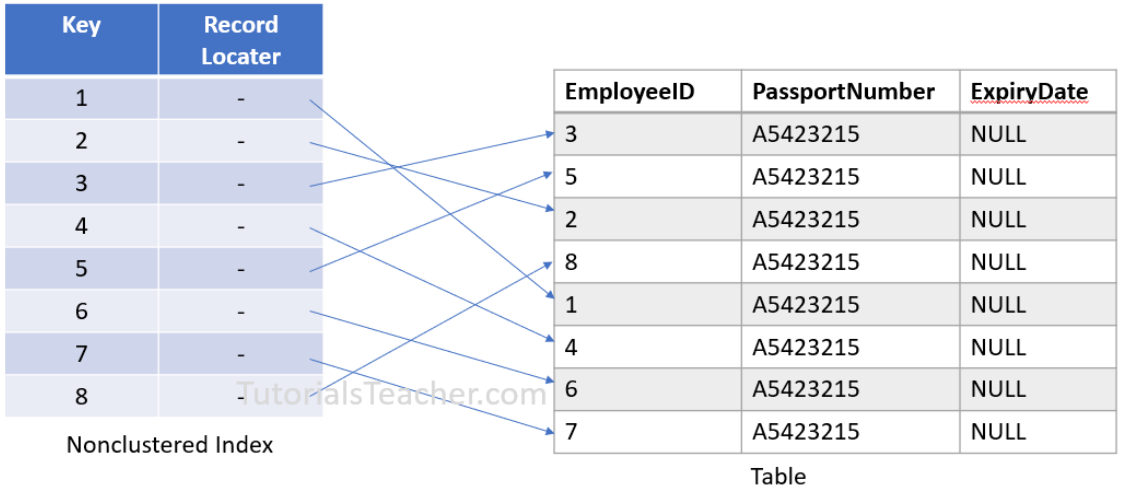
Tổng quan về Lưu trữ và Chỉ mục

- Index là gì?
  - Index là cấu trúc dữ liệu đặc biệt được liên kết với các bảng hoặc Views giúp tăng tốc độ truy vấn.
- Phân loại:
  - Clustered index: Tổ chức dữ liệu bằng cách sử dụng một cây có cấu trúc đặc biệt được gọi là B-tree (Balance Tree) cho phép tìm kiếm, chèn, cập nhật, xóa theo thời gian phân bổ logarit
  - Non-Clustered index : Tổ chức và lưu trữ dữ liệu riêng biệt với các hàng có trong bảng. Nó là một bản sao của các cột dữ liệu đã chọn từ một bảng với các liên kết đến bảng được liên kết.
- Khi nào nên sử dụng Index?
  - Chỉ nên sử dụng index khi cần search thông tin theo cột đó.
- Khi nào không nên sử dụng index?
  - Không nên sử dụng trong các bảng nhỏ, ít bản ghi
  - Không nên sử dụng index cho các cột thường xuyên bị sửa đổi, update dữ liệu

Tổng quan về Lưu trữ và Chỉ mục

- Phân loại: Về cơ bản index phân thành 2 loại là Clustered Index và Non-Clustered Index
  - Clustered Index
  - Non-Clustered Index





Tổng quan về Lưu trữ và Chỉ mục

➢ **Các câu lệnh:**

- Tạo Index:

```
CREATE INDEX index_name
ON table_name (column1, column2, ...);
```

- Đổi tên Index

```
EXEC sp_rename
   index_name,
   new_index_name,
   N'INDEX';
```

```
EXEC sp_rename
   @objname = N'index_name',
   @newname = N'new_index_name',
   @objtype = N'INDEX';
```

- Câu lệnh xóa Index:

```
DROP INDEX table_name.index_name;
```

Tổng quan về Lưu trữ và Chỉ mục

- Vô hiệu hóa 1 Index: Trình tối ưu hóa truy vấn sẽ không xem xét chỉ mục vô hiệu hóa đó để tạo kế hoạch thực thi truy vấn.

```
ALTER INDEX index_name ON table_name DISABLE;
```

- Vô hiệu hóa tất cả Index của 1 bảng:

```
ALTER INDEX index_name ON table_name DISABLE;
```

- Bỏ vô hiệu hóa Index

```
ALTER INDEX index_name ON table_name REBUILD;
```

```
CREATE INDEX index_name ON table_name(column_list)
WITH(DROP_EXISTING=ON)
```

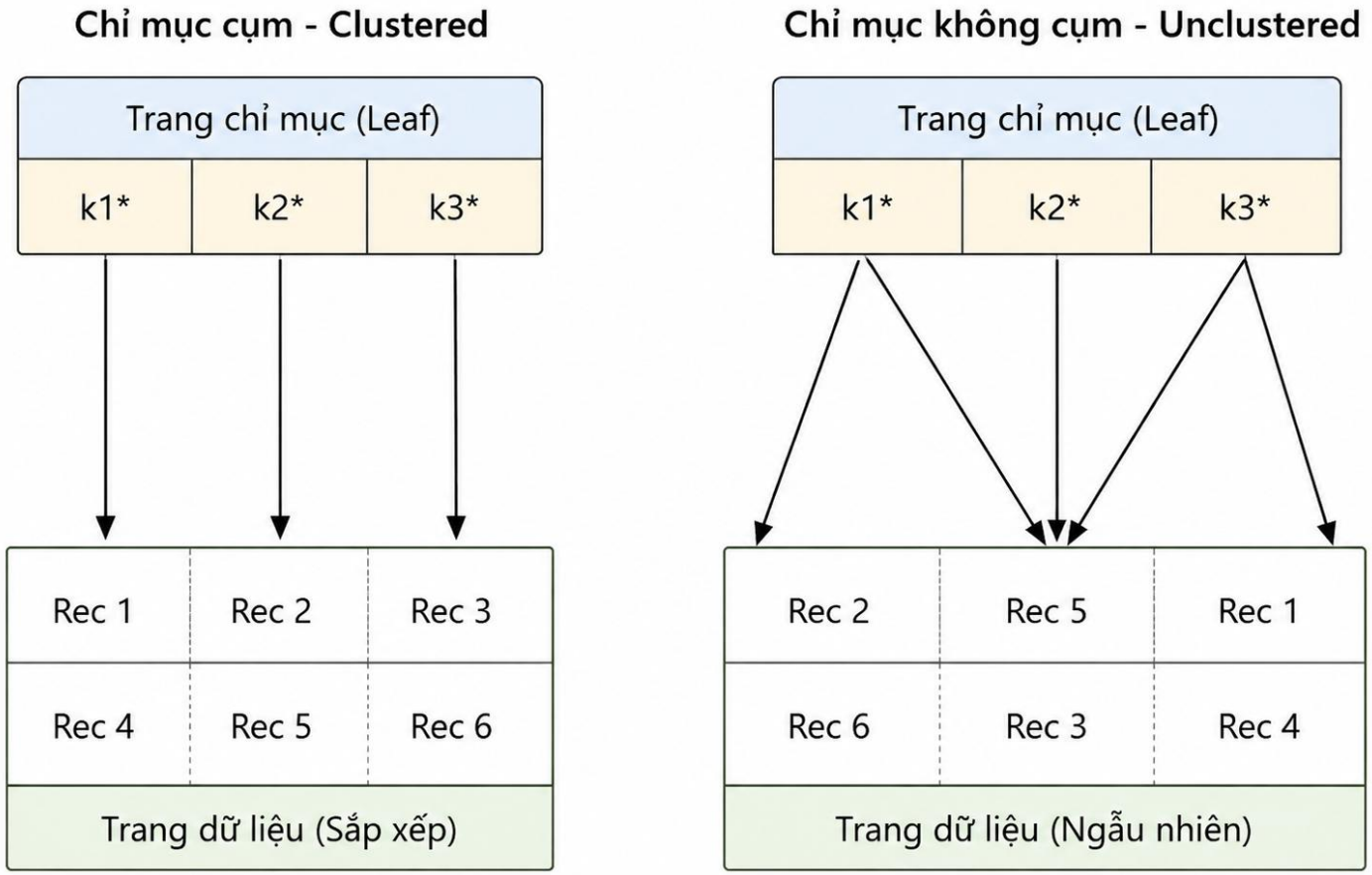
```
ALTER INDEX ALL ON table_name REBUILD;
```

Tổng quan về Lưu trữ và Chỉ mục

- Table Scan:
  - Table Scan xảy ra khi select dữ liệu ở một table không có clustered index
  - Performance thấp nhất
- Index Scan:
  - Index Scan xẩy ra phụ thuộc vào mệnh đề where
  - Performance tốt hơn table scan
- Index Seek:
  - Một table với index được truy cập và cấu trúc B-tree có thể làm lọc được các kết quả trả về, giúp lấy được 1 số lượng nhất định rows từ table
  - Performance tốt nhất

Tổng quan về Lưu trữ và Chỉ mục

Chỉ mục cụm (Clustered Index) và Chỉ mục không cụm (Unclustered Index)



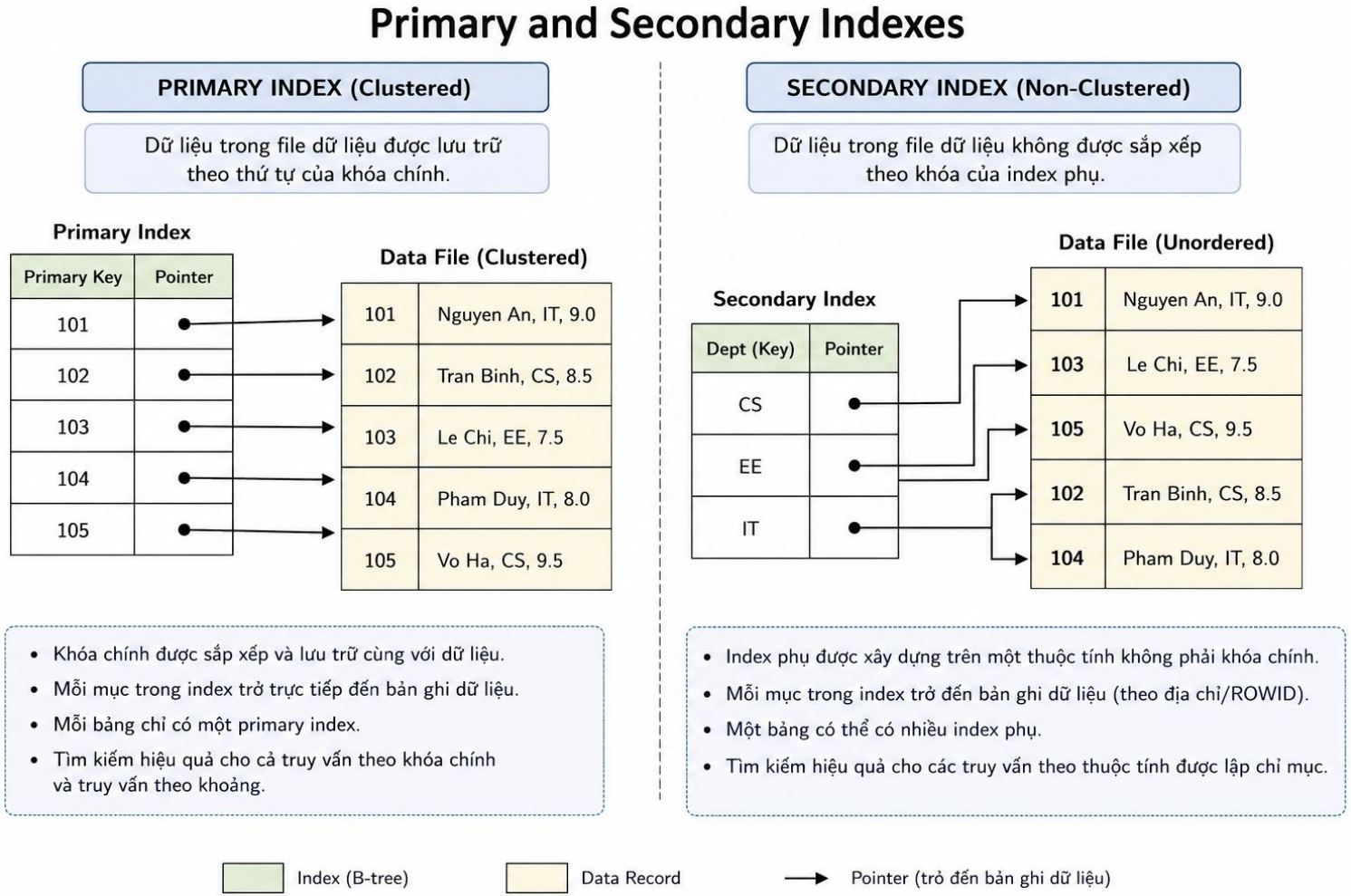
Tổng quan về Lưu trữ và Chỉ mục

Chỉ mục chính (Primary Index) và Chỉ mục phụ (Secondary Index)

- **Chỉ mục chính (Primary Index):**
  - Là chỉ mục được xây dựng trên một trường (hoặc tổ hợp trường) chứa khóa chính của bảng. Đảm bảo các khóa tìm kiếm trong chỉ mục là duy nhất (Unique).
- **Chỉ mục phụ (Secondary Index):**
  - Là chỉ mục xây dựng trên các trường không phải khóa chính. Thường chứa các giá trị khóa tìm kiếm trùng lặp.

Tổng quan về Lưu trữ và Chỉ mục

Chỉ mục chính (Primary Index) và Chỉ mục phụ (Secondary Index)



Tổng quan về Lưu trữ và Chỉ mục

Chỉ mục chính (Primary Index) và Chỉ mục phụ (Secondary Index)

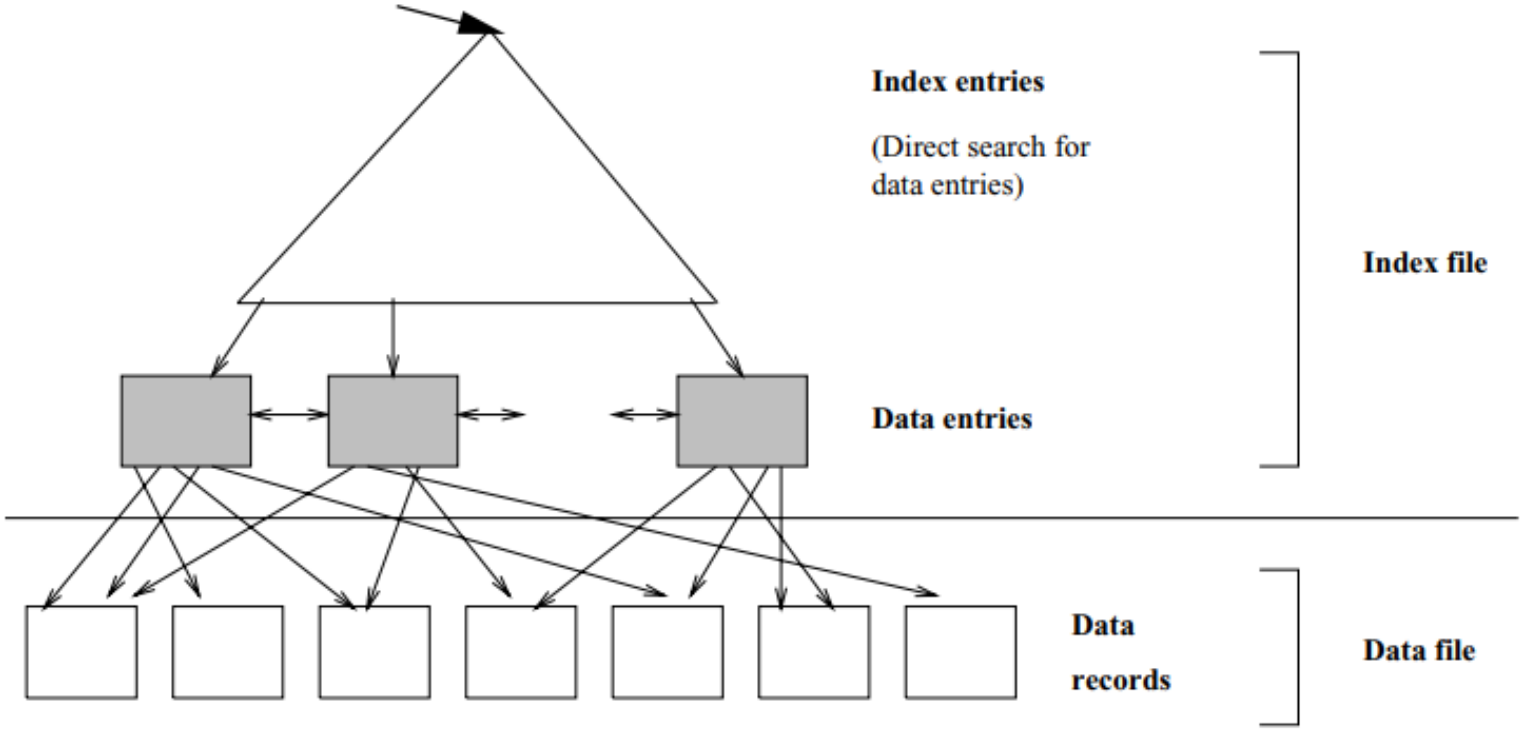


Figure 8.1 Unclustered Index Using Alternative (2)

Chỉ mục dựa trên cấu trúc cây (Tree-Structured Indexing)

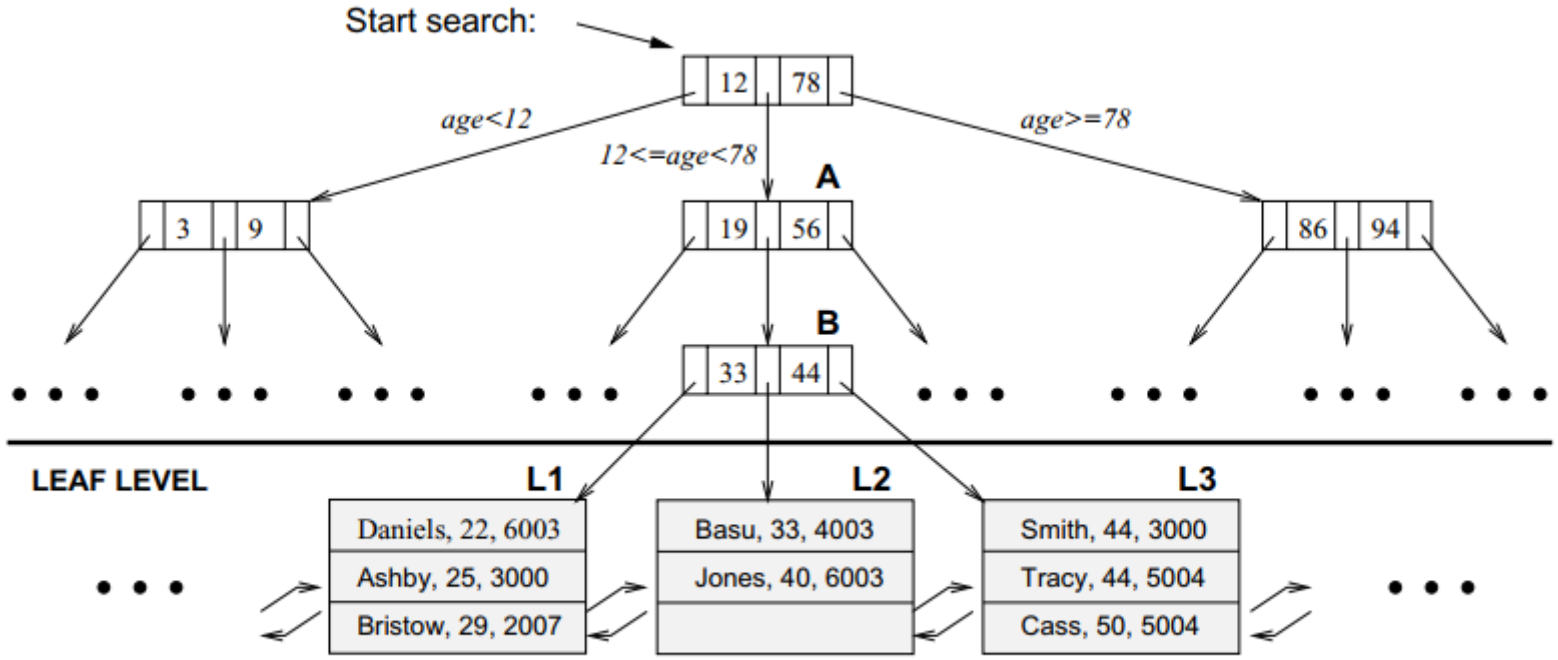


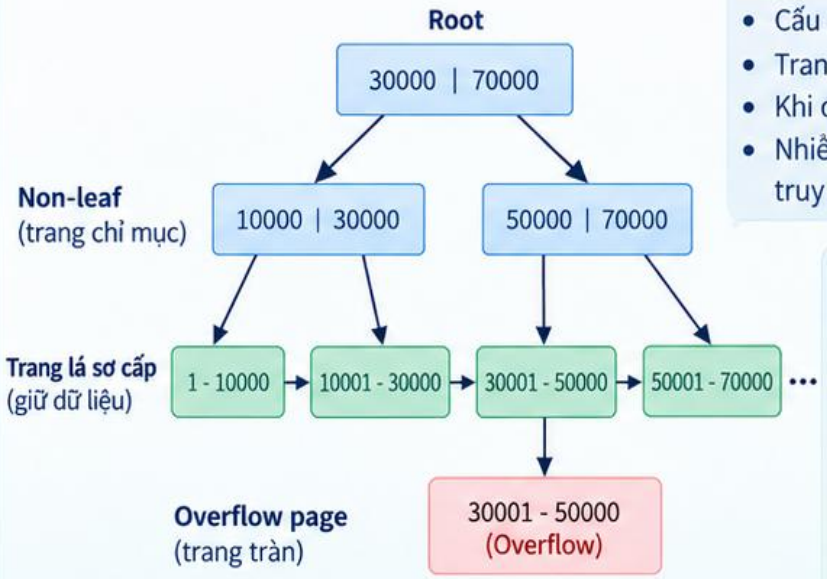
Figure 8.3 Tree-Structured Index

Chỉ mục dựa trên cấu trúc cây (Tree-Structured Indexing)

- Chỉ mục cấu trúc cây được thiết kế để tối ưu hóa phép tìm kiếm khoảng (Range Search) cũng như tìm kiếm bằng nhau (Equality Search)
- Có 2 phương pháp về chỉ mục:
  - **Phương pháp tuần tự chỉ mục (ISAM - Indexed Sequential Access Method)**
  - **Cây B+ (B+ Trees) - Cấu trúc chỉ mục động (Dynamic Index)**

Chỉ mục dựa trên cấu trúc cây (Tree-Structured Indexing)

1 ISAM (Indexed Sequential Access Method)



- Cấu trúc chỉ mục **tĩnh**
- Trang lá sơ cấp được ghi tuần tự
- Khi đầy -> dùng overflow page
- Nhiều overflow chain -> truy xuất chậm

Ví dụ tìm MASV = 45000

Root: 30000 | 70000

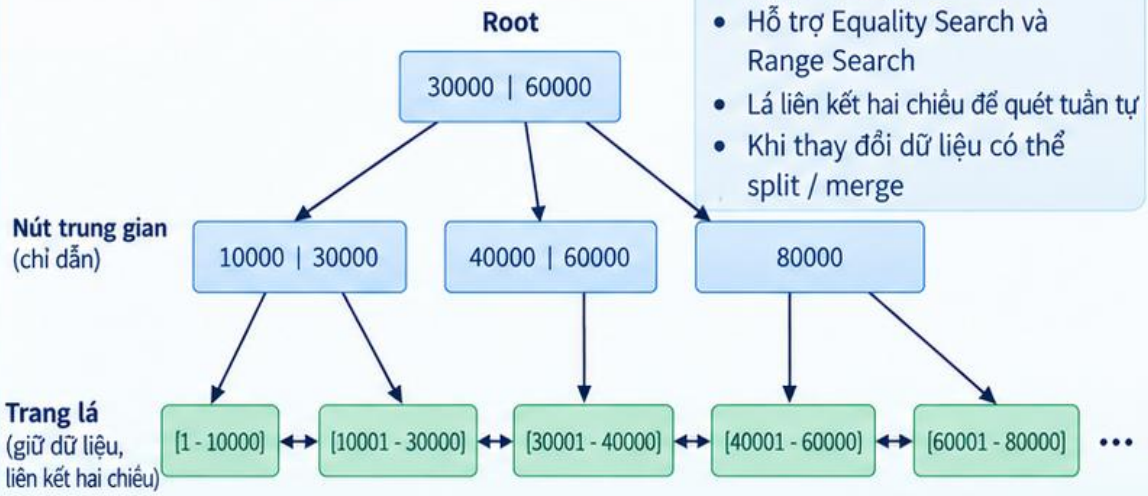
↓

Non-leaf: 30001 | 50000

↓

Lá: 30001 - 50000
(đọc trang và trang tràn nếu có)

2 B+ Tree



- Tự cân bằng
- Hỗ trợ Equality Search và Range Search
- Lá liên kết hai chiều để quét tuần tự
- Khi thay đổi dữ liệu có thể split / merge

Ví dụ 1: Truy vấn tìm kiếm bằng giá trị (Equality Search)

```
1  SELECT *
2  FROM dbo.SINHVIEN
3  WHERE MASV = 90000;
```

ISAM

Duyệt theo cây đến trang lá chứa MASV = 90000 (đọc thêm overflow nếu có).

B+ Tree

Duyệt theo cây đến trang lá chứa MASV = 90000 (hiệu quả, không cần đò overflow).

Ví dụ 2: Truy vấn khoảng (Range Search)

```
1  SELECT *
2  FROM dbo.SINHVIEN
3  WHERE MASV BETWEEN
4   50000 AND 50100;
```

B+ Tree rất phù hợp cho truy vấn khoảng

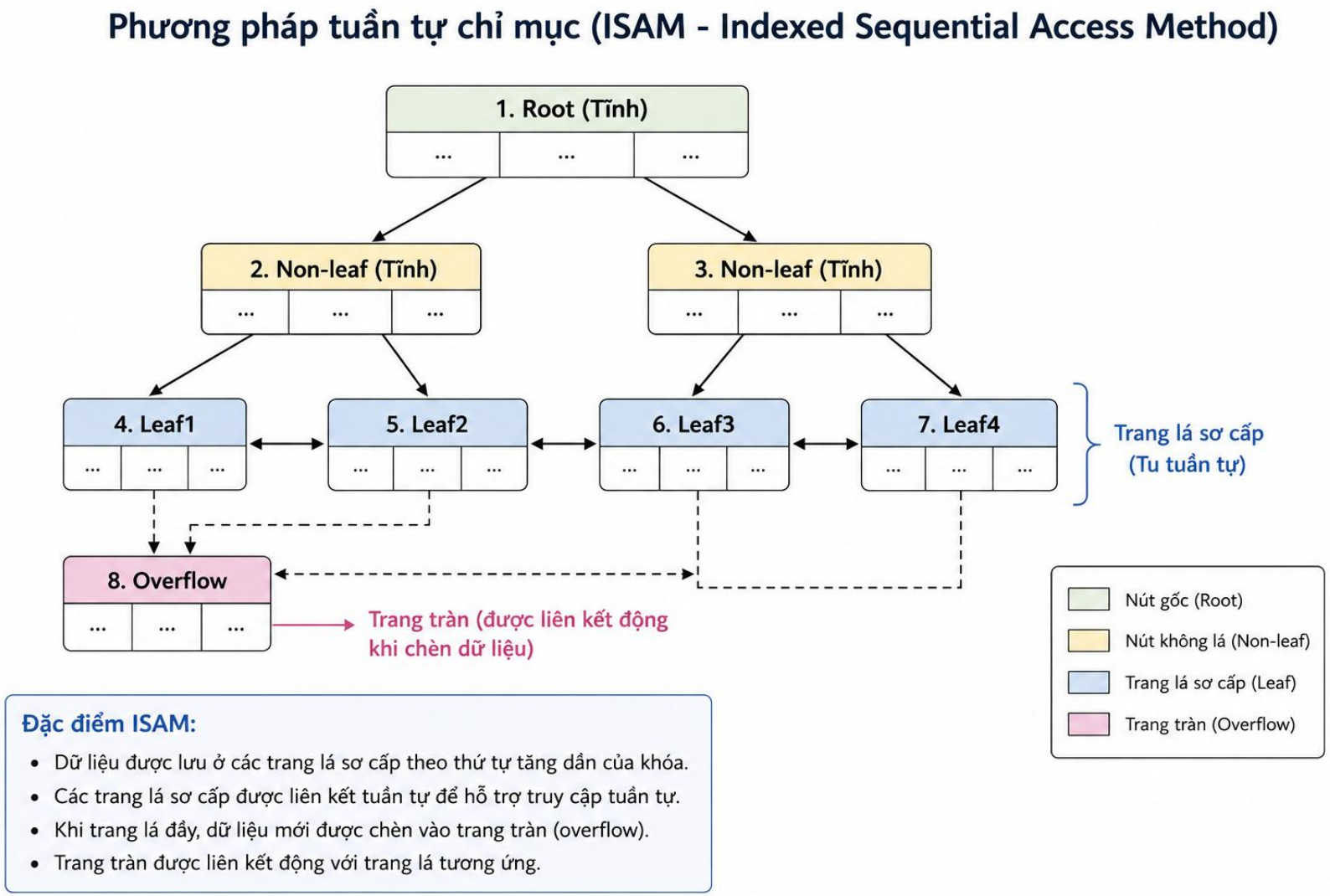
Tìm trang lá đầu tiên (>= 50000), sau đó quét tuần tự qua các trang lá liên kết cho đến khi > 50100.



Chỉ mục dựa trên cấu trúc cây (Tree-Structured Indexing)

Phương pháp tuần tự chỉ mục (ISAM - Indexed Sequential Access Method)

- ISAM là cấu trúc cây chỉ mục tĩnh (static), dữ liệu lá được phân bổ vật lý tuần tự khi khởi tạo tệp.



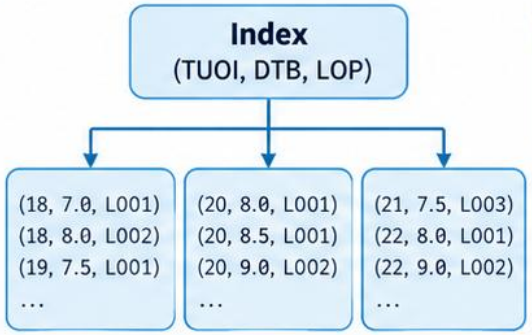
Chỉ mục dựa trên cấu trúc cây (Tree-Structured Indexing)

Phương pháp tuần tự chỉ mục (ISAM - Indexed Sequential Access Method)

- **Đặc điểm cấu trúc:**
  - Các trang không phải lá (Non-leaf nodes) đóng vai trò điều hướng. Chúng chứa các cặp [Search Key, Page Pointer].
  - Các trang lá sơ cấp (Primary Leaf Pages) được ghi liên tiếp nhau trên đĩa nhằm tối ưu hóa các phép quét tuần tự.
- **Thao tác tìm kiếm (Search):**
  - Bắt đầu từ gốc, so sánh khóa tìm kiếm với các khóa phân tách ở nút trung gian để chọn nhánh con, đi thẳng xuống trang lá tương ứng.

Chú ý

1 Composite index (chỉ mục nhiều cột)



Tạo composite index:

```
CREATE INDEX IX_SINHVIEN_TUOI_DTB_LOP
ON dbo.SINHVIEN(TUOI, DTB, LOP);
```

Thứ tự so sánh:

1. So sánh theo TUOI;
2. Nếu TUOI bằng nhau, so sánh theo DTB;
3. Nếu TUOI và DTB bằng nhau, so sánh theo LOP.

2 Quy tắc khớp chỉ mục (Index Matching Rule)

Truy vấn (WHERE)	Có khớp chỉ mục?
`WHERE TUOI = 20`	✓ Có
`WHERE TUOI = 20 AND DTB >= 8`	✓ Có
`WHERE TUOI = 20 AND DTB >= 8 AND LOP = 'L001'`	✓ Có
`WHERE DTB = 8.5`	✗ Không
`WHERE LOP = 'L001'`	✗ Không

Index cây thường khớp theo tiền tố liên tục (leftmost prefix).

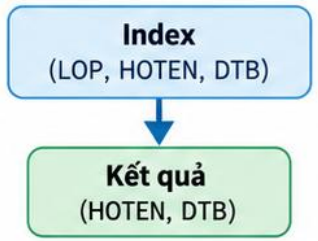
3 Index-Only Plan / Covering Index

Tạo covering index (dùng INCLUDE):

```
CREATE INDEX IX_SINHVIEN_LOP_COVER
ON dbo.SINHVIEN(LOP) INCLUDE(HOTEN, DTB);
```

Truy vấn sử dụng index-only plan:

```
SELECT HOTEN, DTB
FROM dbo.SINHVIEN
WHERE LOP = 'L007';
```



Lấy dữ liệu trực tiếp từ index (không cần đọc bảng gốc).

4 Bài tập nhanh cho sinh viên

1. Truy vấn nào nên dùng B+ Tree?
2. Khi nào Hash phù hợp hơn?
3. Vì sao composite index (TUOI, DTB, LOP) không tối ưu cho WHERE DTB = 8.5?

Index không phải càng nhiều càng tốt; hãy tạo index theo truy vấn thực tế.

Chỉ mục dựa trên cấu trúc cây (Tree-Structured Indexing)

Phương pháp tuần tự chỉ mục (ISAM - Indexed Sequential Access Method)

- **Thao tác chèn và xóa (Insert & Delete):**
  - Vì các trang lá sơ cấp được cố định vật lý, khi một trang lá bị đầy do thao tác chèn, hệ thống sẽ cấp phát một trang mới từ **vùng tràn (overflow area)** và xâu chuỗi nó vào trang lá sơ cấp.
  - *Hạn chế:* Nếu dữ liệu chèn tập trung vào một vùng, các chuỗi trang tràn (overflow chains) sẽ kéo dài, làm giảm hiệu năng tìm kiếm nghiêm trọng (phải quét tuyến tính chuỗi tràn). Hệ thống cần phải tái cấu trúc toàn bộ tệp (reorganization) để khôi phục hiệu năng.
- **Lợi thế về Truy cập đồng thời (Concurrency Control):**
  - Vì các nút điều hướng trung gian không bao giờ bị thay đổi cấu trúc sau khi khởi tạo, các giao dịch đọc dữ liệu không cần khóa (lock) các trang trung gian này. Điều này giúp ISAM có ưu thế vượt trội về mặt kiểm soát truy cập đồng thời so với Cây B+ khi dữ liệu ít biến động.

Chỉ mục dựa trên cấu trúc cây (Tree-Structured Indexing)

Cây B+ (B+ Trees) - Cấu trúc chỉ mục động (Dynamic Index)

- Cây B+ là cấu trúc cây tự cân bằng động, khắc phục triệt để nhược điểm của ISAM bằng cách tự động điều chỉnh cấu trúc (phân tách nút, gộp nút) khi dữ liệu thay đổi.
- **Đặc trưng kỹ thuật của Cây B+:**
  - **Tính cân bằng hoàn hảo:** Mọi đường đi từ nút gốc (root) đến bất kỳ nút lá (leaf) nào đều có độ dài bằng nhau (gọi là chiều cao *h* của cây).
  - **Định dạng nút (Node Format):**
    - Mỗi nút (một trang đĩa) có bậc (order) ký hiệu là *d*.
    - Mọi nút (trừ nút gốc) bắt buộc phải chứa *m* thực thể dữ liệu sao cho: $d \leq m \leq 2d$
    - Tương đương với việc đảm bảo hiệu suất sử dụng không gian đĩa tối thiểu là **50%**.
    - Nút gốc là ngoại lệ duy nhất, có thể chứa từ 1 đến 2*d* thực thể.
  - **Nút lá (Leaf Nodes):** Chứa các thực thể dữ liệu chỉ mục thực tế k^*. Các nút lá được liên kết hai chiều (doubly-linked list) tạo thành **tập tuần tự (sequence set)** để hỗ trợ quét dữ liệu cực kỳ hiệu quả theo cả hai chiều.

Chỉ mục dựa trên cấu trúc cây (Tree-Structured Indexing)

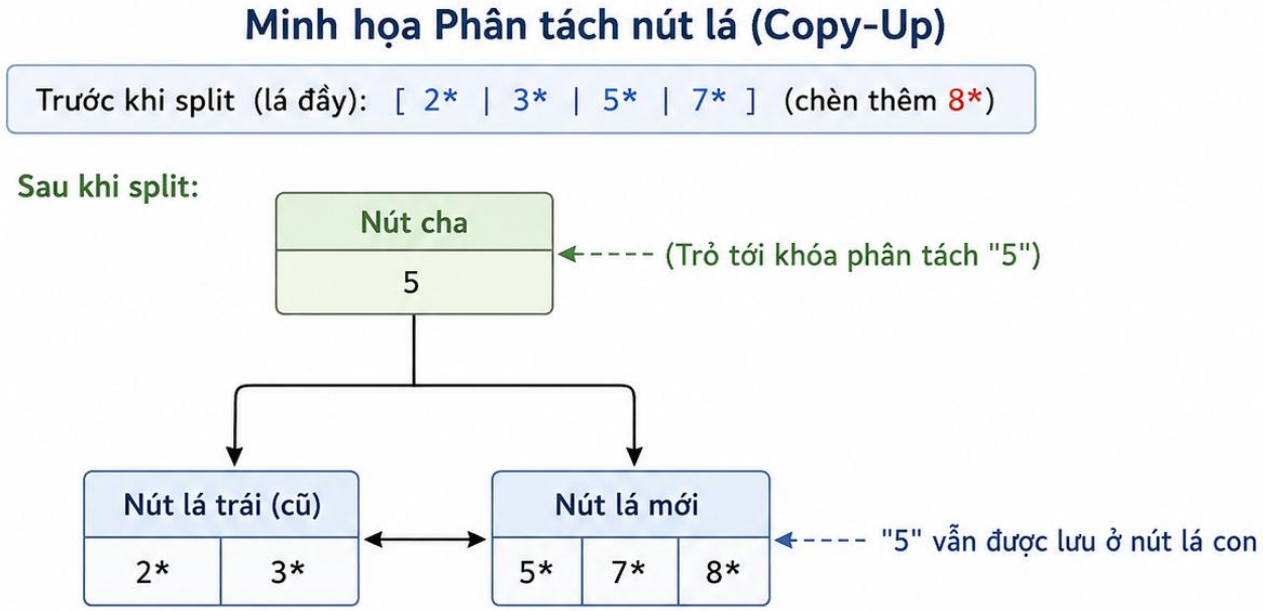
Thuật toán Chèn (Insertion) vào Cây B+:

- Khi chèn một thực thể dữ liệu *entry* vào nút lá *L*:
- **Trường hợp 1: Nút lá *L* còn chỗ trống (*m*<2*d*):** Chèn *entry* vào nút lá theo đúng trật tự sắp xếp.

Chỉ mục dựa trên cấu trúc cây (Tree-Structured Indexing)

Thuật toán Chèn (Insertion) vào Cây B+:

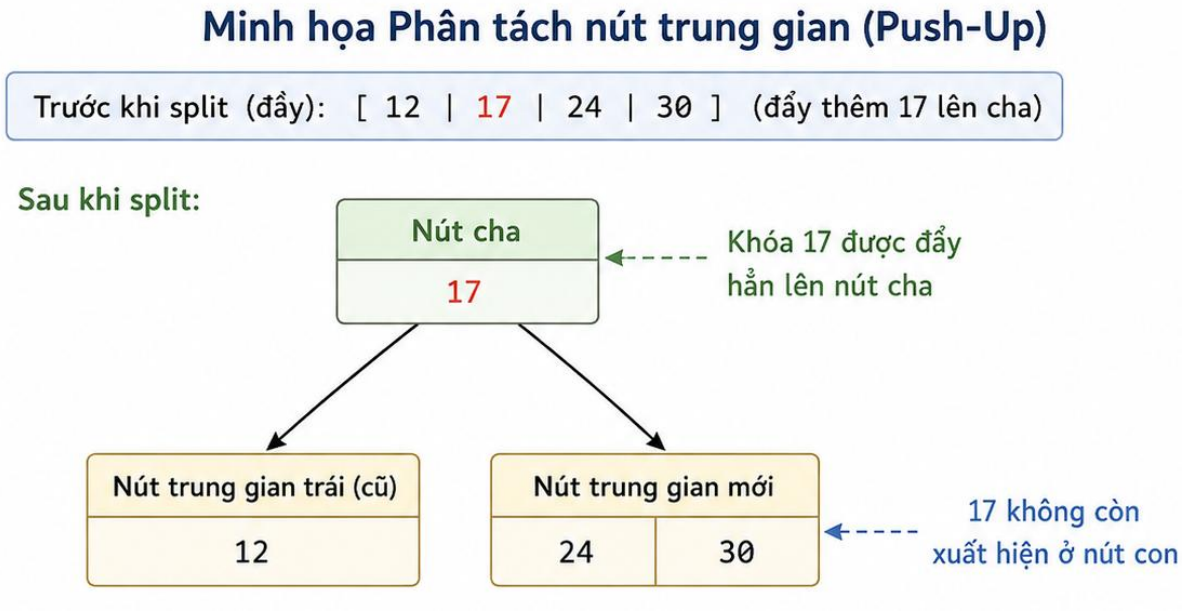
- **Trường hợp 2: Nút lá *L* đã đầy (*m*=2*d*):**
  - Tiến hành **Phân tách nút lá (Split Leaf)**: Tạo ra một nút lá mới *L2*.
  - Phân phối lại 2*d*+1 thực thể dữ liệu: *d* phần tử đầu tiên giữ lại ở *L*, *d*+1 phần tử còn lại chuyển sang nút mới *L2*.
  - Sao chép giá trị khóa nhỏ nhất của *L2* lên nút cha (cơ chế **Copy-Up**) để làm phần tử phân tách điều hướng. Bản ghi này vẫn tiếp tục xuất hiện ở nút lá *L2*.



2.2. Chỉ mục dựa trên cấu trúc cây (Tree-Structured Indexing)

Thuật toán Chèn (Insertion) vào Cây B+:

- **Trường hợp 3: Phân tách nút trung gian (Split Non-leaf):**
  - Nếu nút cha cũng đầy, quá trình phân tách được thực hiện đệ quy ngược lên trên.
  - Khi phân tách nút trung gian chứa $2d$ khóa và $2d+1$ con trỏ: Tạo nút trung gian mới $N2$. Giữ d khóa ở nút cũ, chuyển d khóa sang $N2$, và khóa ở giữa (khóa thứ $d+1$) sẽ được đẩy hẳn lên nút cha cao hơn (cơ chế **Push-Up**). Khóa này **không** còn xuất hiện ở nút con nữa.
  - *Tăng chiều cao cây:* Nếu nút gốc bị phân tách, một nút gốc mới được tạo ra với một khóa phân tách duy nhất. Đây là cách duy nhất chiều cao của Cây B+ tăng lên.



Chỉ mục dựa trên cấu trúc cây (Tree-Structured Indexing)

Thuật toán Xóa (Deletion) trên Cây B+:

- Khi xóa một thực thể dữ liệu chỉ mục khỏi nút *N*:

1. Nếu nút *N* sau khi xóa vẫn thỏa mãn điều kiện số lượng phần tử tối thiểu ($m \geq d$): Kết thúc thao tác.
2. Nếu nút *N* bị rơi vào trạng thái thiếu phần tử (underflow, $m < d$):
   - **Phân phối lại (Redistribution):** Kiểm tra các nút anh em kề cận (siblings) có cùng nút cha. Nếu nút anh em có dư phần tử ($m > d$), tiến hành mượn phần tử từ nút anh em thông qua việc điều chỉnh khóa phân tách ở nút cha.
   - **Gộp nút (Merge Nodes):** Nếu tất cả các nút anh em kề cận cũng chỉ có đúng số lượng phần tử tối thiểu ($m = d$), tiến hành gộp nút *N* với nút anh em đó. Sau khi gộp, ta phải xóa khóa phân tách tương ứng trên nút cha. Quá trình này có thể lan truyền đệ quy lên trên và làm giảm chiều cao của cây đi 1 mức nếu nút gốc bị triệt tiêu.

Chỉ mục dựa trên phương pháp băm (Hash-based indexing)

Chỉ mục dựa trên phương pháp băm (Hash-based indexing)

- Chỉ mục băm là giải pháp tối ưu tuyệt đối cho các truy vấn tìm kiếm bằng nhau (Equality Search - $O(1)$) nhưng không hỗ trợ các phép tìm kiếm khoảng (Range Search).
- Các phương pháp băm:
  - **Băm tĩnh (Static Hashing)**
  - **Băm mở rộng (Extendible Hashing)**
  - **Băm tuyến tính (Linear Hashing)**

Chỉ mục dựa trên phương pháp băm (Hash-based indexing)

Băm tĩnh (Static Hashing)

- **Cấu trúc:** Gồm một số lượng cố định các **Buckets** (Thùng chứa) được đánh số từ 0 đến $N-1$. Mỗi bucket gồm một trang đĩa chính (Primary Page) và có thể liên kết thêm các trang tràn (Overflow Pages).
- **Hàm băm (h):** Ánh xạ giá trị khóa tìm kiếm *key* vào dải số nguyên $[0, N-1]$:

$$h(key) \pmod N = \text{Mã số Bucket}$$

- **Hạn chế:** Nếu số lượng dữ liệu tăng lên quá lớn so với thiết kế ban đầu (N), các chuỗi trang tràn sẽ kéo dài làm giảm hiệu năng hệ thống. Nếu chọn N quá lớn từ đầu sẽ gây lãng phí tài nguyên.

Chỉ mục dựa trên phương pháp băm (Hash-based indexing)

Băm tĩnh (Static Hashing)

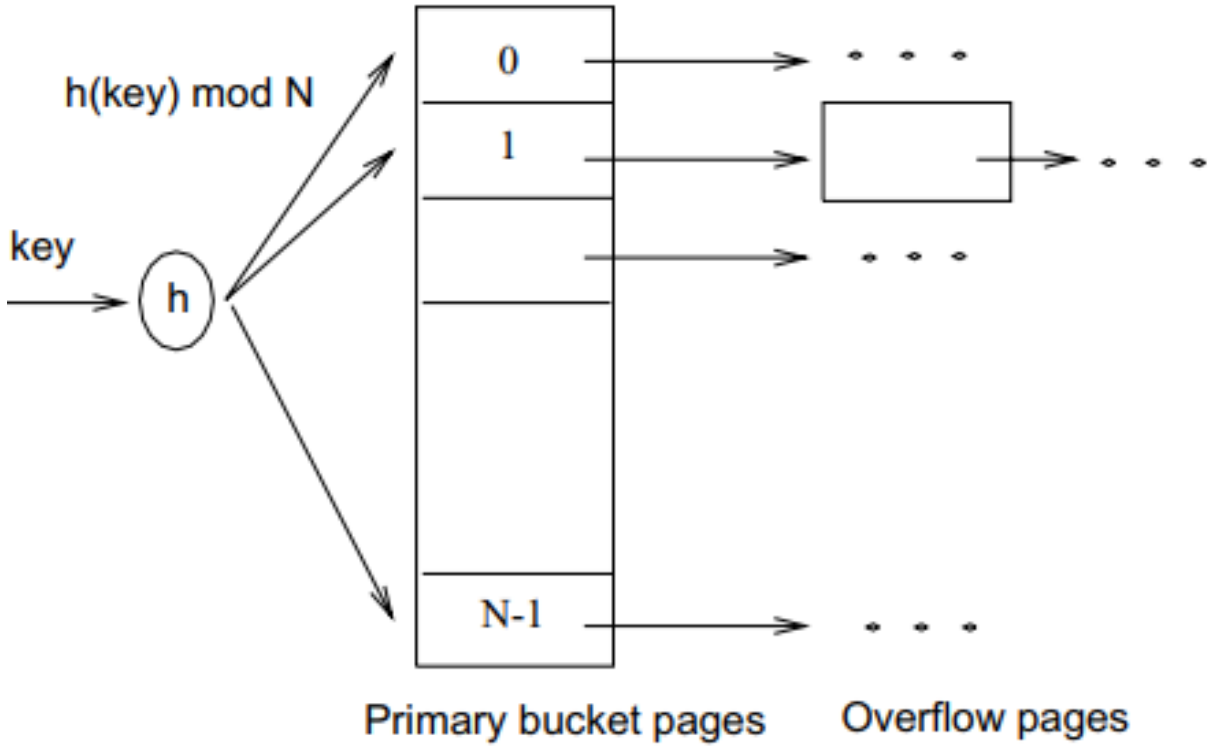


Figure 11.1 Static Hashing

Chỉ mục dựa trên phương pháp băm (Hash-based indexing)

- **Băm mở rộng (Extendible Hashing)**
  - Băm mở rộng giải quyết vấn đề của băm tĩnh bằng cách sử dụng một **Thư mục chứa các con trỏ (Directory of Pointers)** trỏ đến các bucket dữ liệu thực tế và cho phép thư mục này nhân đôi kích thước một cách linh hoạt.

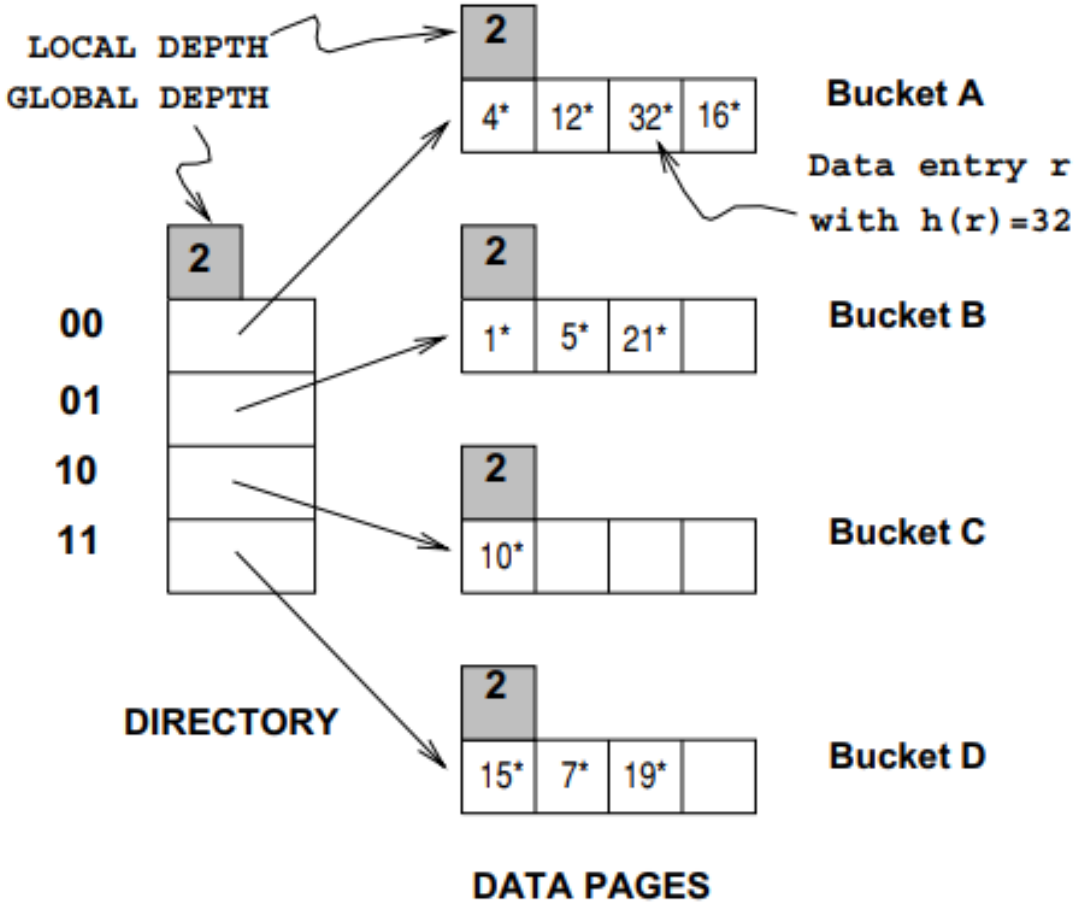


Figure 11.2 Example of an Extendible Hashed File

Chỉ mục dựa trên phương pháp băm (Hash-based indexing)

Băm mở rộng (Extendible Hashing)

- **Độ sâu toàn cục (Global Depth - *d*):**
  - Thể hiện kích thước của thư mục hiện tại là 2^d phần tử. Hệ thống sẽ sử dụng *d* bit cuối của giá trị băm *h*(*key*) để làm chỉ mục tìm kiếm trong thư mục.
- **Độ sâu cục bộ (Local Depth - ℓ_i):**
  - Mỗi bucket dữ liệu *i* có một độ sâu cục bộ ℓ_i ($\ell_i \leq d$). Nó biểu thị rằng mọi phần tử trong bucket này đều có chung ℓ_i bit cuối giống nhau. Số lượng phần tử thư mục cùng trỏ tới bucket *i* này là $2^{d-\ell_i}$.

Chỉ mục dựa trên phương pháp băm (Hash-based indexing)

Băm mở rộng (Extendible Hashing)

- **Thao tác chèn và Phân tách Bucket (Split Bucket):**
  - Khi chèn phần tử vào một bucket đã đầy: Tiến hành phân tách bucket thành 2 bucket mới, đồng thời phân phối lại dữ liệu dựa trên giá trị của bit thứ ℓ_i +1 tính từ cuối. Tăng độ sâu cục bộ ℓ_i của bucket lên thành ℓ_i +1.
  - **Nhân đôi thư mục (Directory Doubling):**
    - Nếu ℓ_i ban đầu của bucket bị đầy **nhỏ hơn** Global Depth *d*: Ta chỉ cần tạo một bucket mới, tăng ℓ_i lên và trỏ lại các con trỏ tương ứng trong thư mục mà không cần nhân đôi thư mục.
    - Nếu ℓ_i ban đầu **bằng** Global Depth *d*: Ta bắt buộc phải **nhân đôi thư mục** (Global Depth tăng lên thành *d*+1). Thư mục mới có kích thước gấp đôi, các con trỏ được sao chép tương ứng và chỉ có bucket bị đầy được phân tách thực sự.

Chỉ mục dựa trên phương pháp băm (Hash-based indexing)

Băm tuyến tính (Linear Hashing)

- Băm tuyến tính là giải pháp băm động loại bỏ hoàn toàn việc sử dụng thư mục con trỏ. Thay vào đó, nó thực hiện phân tách các bucket theo một trật tự tuần tuần tự (round-robin) cố định.
- **Các biến trạng thái chính:**
  - *Level*: Vòng phân tách hiện tại (bắt đầu từ 0).
  - *Next*: Con trỏ xác định bucket tiếp theo sẽ bị phân tách trong vòng này (bắt đầu từ 0).
  - N_{Level}: Số lượng bucket tại thời điểm bắt đầu vòng *Level*: $N_{Level} = N_0 \times 2^{Level}$

Chỉ mục dựa trên phương pháp băm (Hash-based indexing)

Băm tuyến tính (Linear Hashing)

- **Họ các hàm băm (h_{Level}):** $h_i(key)=h(key) \pmod{N_0 \times 2^i}$ Hàm băm của vòng sau ($h_{Level+1}$) luôn có dải giá trị gấp đôi hàm băm vòng trước (h_{Level}).
- **Cơ chế tìm kiếm (Search):** Áp dụng hàm băm vòng hiện tại $h_{Level}(key)=b$:
  - Nếu $b \geq Next$: Bucket này chưa bị phân tách trong vòng này. Bản ghi nằm ở bucket b.
  - Nếu $b < Next$: Bucket b đã bị phân tách trước đó. Áp dụng tiếp hàm băm vòng kế tiếp $h_{Level+1}(key)$ để xác định bản ghi nằm ở bucket b hay bucket ảnh của nó là $b+N_{Level}$.

Chỉ mục dựa trên phương pháp băm (Hash-based indexing)

Băm tuyến tính (Linear Hashing)

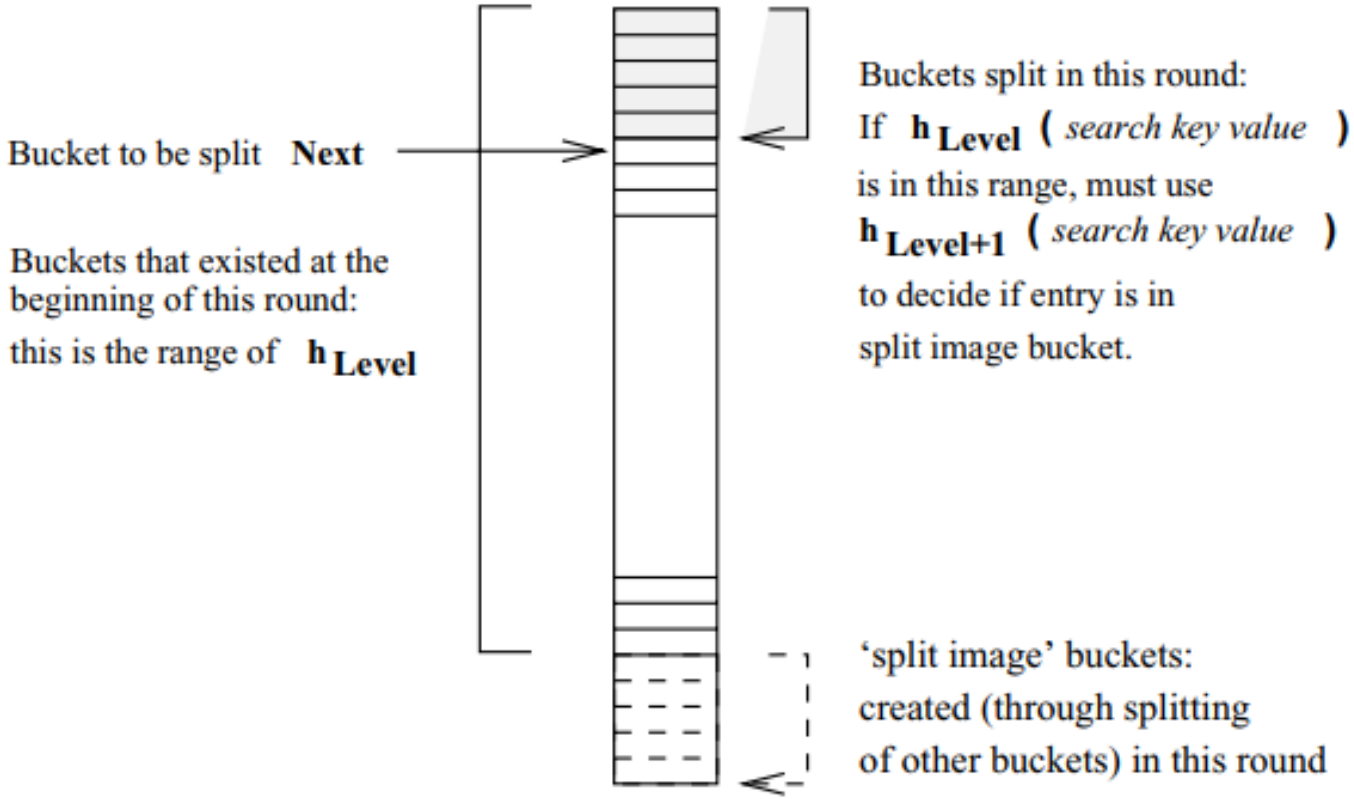


Figure 11.7 Buckets during a Round in Linear Hashing

Chỉ mục dựa trên phương pháp băm (Hash-based indexing)

Băm tuyến tính (Linear Hashing)

- **Cơ chế chèn và phân tách tuần tự:**
  - Khi có phần tử chèn làm phát sinh thêm trang tràn (overflow page), hệ thống kích hoạt một lần phân tách.
  - *Điểm đặc biệt:* Bucket bị phân tách vật lý luôn là bucket được trỏ bởi con trỏ *Next*, chứ **không nhất thiết** là bucket vừa nhận dữ liệu chèn.
  - Phần tử của bucket *Next* được phân bổ lại giữa chính nó và bucket ảnh mới *Next*+ N_{Level} bằng hàm băm $h_{Level+1}$. Sau đó, *Next* được dịch chuyển lên 1 đơn vị (*Next* = *Next*+1).
  - *Kết thúc vòng:* Khi *Next* đạt tới giá trị $N_{Level} - 1$ và bị phân tách, toàn bộ các bucket ban đầu đều đã được phân tách xong. Vòng phân tách kết thúc, hệ thống tăng *Level*=*Level*+1 và đặt lại *Next*=0.

So sánh chi phí I/O của các tổ chức tệp và chỉ mục

So sánh chi phí I/O của các tổ chức tệp và chỉ mục

- **Mô tả Tham số và Mô hình Chi phí (Cost Model)**
  - *B*: Số lượng trang dữ liệu (data pages) khi các bản ghi được xếp khít nhau không lãng phí.
  - *R*: Số lượng bản ghi dữ liệu trên một trang (records per page).
  - *D*: Thời gian trung bình để đọc hoặc ghi một trang đĩa vật lý (Page I/O).
  - *C*: Chi phí CPU trung bình để xử lý một bản ghi trong bộ nhớ.
  - *H*: Chi phí áp dụng hàm băm cho một khóa tìm kiếm.
  - *F*: Độ phân nhánh (fan-out) trung bình của các nút trung gian trong cây chỉ mục.
- *Lưu ý về hệ số lấp đầy:*
  - Đối với các tổ chức tệp động như Cây B+ hay Băm động, các trang thường chỉ có tỷ lệ lấp đầy thực tế trung bình là **67%** (đối với cây) hoặc **80%** (đối với băm tĩnh/băm tuyến tính). Do đó, số lượng trang vật lý thực tế của tệp Clustered File sẽ tăng lên tương ứng là 1.5*B*.

So sánh chi phí I/O của các tổ chức tệp và chỉ mục

File Type	*Scan*	*Equality Search*	*Range Search*	*Insert*	*Delete*
Heap	BD	$0.5BD$	BD	$2D$	*Search*+ D
Sorted	BD	$Dlog_2B$	$Dlog_2B$+# *matching pages*	*Search* + BD	*Search*+ BD
Clustered	$1.5BD$	$Dlog_F1.5B$	$Dlog_F1.5B$+# *matching pages*	*Search* + D	*Search*+ D
Unclustered tree index	$BD(R + 0.15)$	$D(1 + log_F0.15B)$	$D(log_F0.15B$+# *matching records*$)$	$D(3 + log_F0.15B)$	*Search*+ $2D$
Unclustered hash index	$BD(R + 0.125)$	$2D$	BD	$4D$	*Search*+ $2D$

Figure 8.4 A Comparison of I/O Costs

Thiết kế vật lý và tinh chỉnh chỉ mục (index selection & tuning)

Thiết kế vật lý và tinh chỉnh chỉ mục (index selection & tuning)

Khóa tìm kiếm phức hợp (Composite Search Keys)

- Một chỉ mục có khóa tìm kiếm chứa nhiều trường dữ liệu được gọi là chỉ mục khóa phức hợp.
  - **Ví dụ:** Một chỉ mục được xây dựng trên thuộc tính ⟨*age*,*sal*⟩.
  - **Trật tự sắp xếp vật lý:** Cực kỳ quan trọng. Dữ liệu được sắp xếp theo thuộc tính thứ nhất (*age*), nếu trùng nhau mới được sắp xếp tiếp theo thuộc tính thứ hai (*sal*).
- **Quy tắc so khớp chỉ mục (Index Matching Rule):**
  - **Đối với chỉ mục Băm phức hợp:** Chỉ khớp với câu truy vấn nếu nó chứa phép so sánh bằng (=) trên **tất cả** các trường của khóa chỉ mục phức hợp.
  - **Đối với cấu trúc Cây phức hợp:** Chỉ khớp nếu câu truy vấn chứa điều kiện (bằng hoặc khoảng) trên một **tiền tố liên tục (prefix)** của khóa phức hợp.
    - *Ví dụ:* Chỉ mục cây trên ⟨*age*,*sal*,*deptid*⟩ sẽ khớp với câu lọc trên *age* hoặc trên ⟨*age*,*sal*⟩, nhưng **không** khớp với câu lọc chỉ trên *sal* hoặc ⟨*sal*,*deptid*⟩.

Khóa tìm kiếm phức hợp (Composite Search Keys)

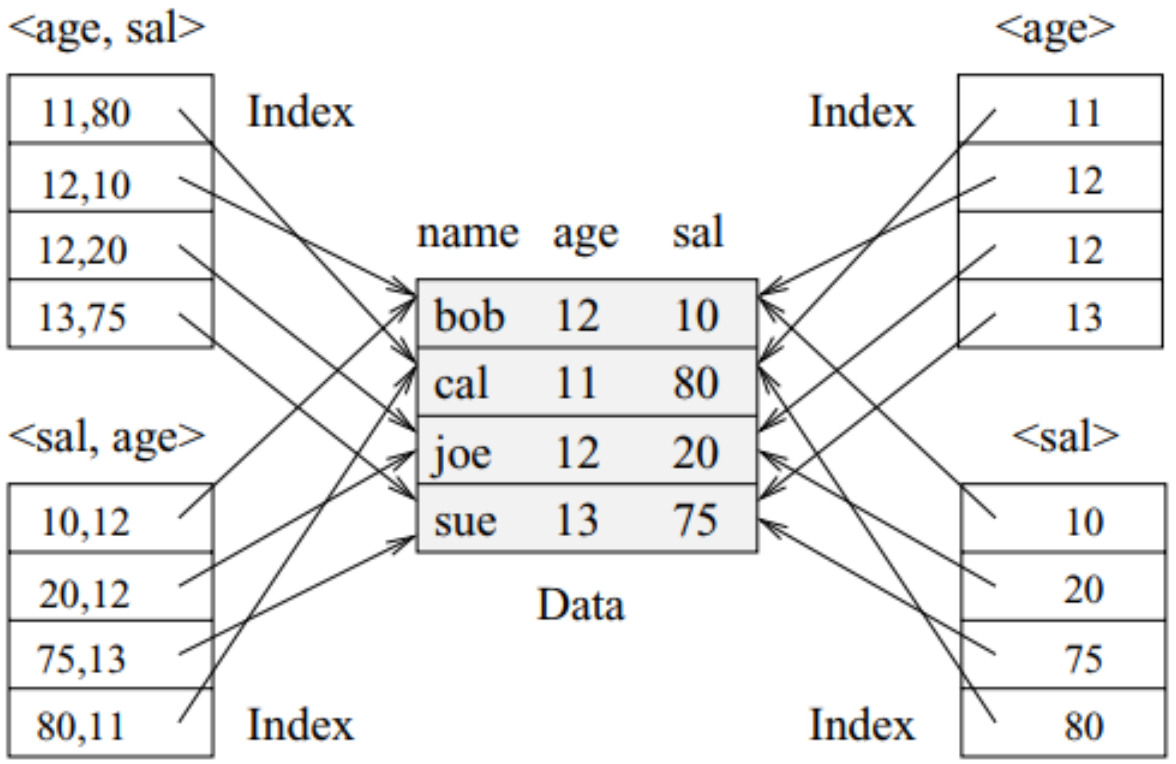


Figure 8.5 Composite Key Indexes

Thiết kế vật lý và tinh chỉnh chỉ mục (index selection & tuning)

Kế hoạch thực thi chỉ truy cập chỉ mục (Index-Only Plans)

- Nếu một câu truy vấn chỉ yêu cầu lấy thông tin từ các thuộc tính nằm hoàn toàn trong khóa tìm kiếm của một chỉ mục, bộ tối ưu hóa truy vấn (Query Optimizer) có thể thực hiện một chiến lược thông minh: **Chỉ quét tệp chỉ mục mà không bao giờ cần truy xuất trang dữ liệu gốc trên đĩa**.
- **Lợi ích vượt trội:**
  - Vì kích thước của thực thể chỉ mục nhỏ hơn nhiều so với bản ghi thực tế, số lượng trang chỉ mục cần đọc ít hơn hẳn.
  - Điều đặc biệt là chiến lược này đạt hiệu quả rất cao ngay cả với **chỉ mục không cụm (unclustered index)** vì hệ thống không bao giờ cần dùng đến các con trỏ RID để thực hiện I/O ngẫu nhiên xuống tệp dữ liệu gốc.

3.6. Bài tập

- 1. Xây dựng SP tính toán giá trị cho đơn hàng có mã cho trước với quan hệ DONHANG(MA, SOLUONG, DONGIA, THUESUAT, CHIETKHAU, THANHTIEN)
- 2. Xây dựng SP tính điểm trung bình và xếp loại cho sinh viên thuộc lớp cho trước. Giả sử có các quan hệ như sau:

  SINHVIEN (MASV, HOTEN, DTB, XEPLOAI, LOP)

  MONHOC (MAMH, TENMH)

  KETQUA (MAMH, MASV, LANTHI, DIEM)

  Trong đó:
  - Điểm thi chỉ tính lần thi sau cùng.
  - Xếp loại: Xuất sắc [9, 10], Giỏi [8, 8.9], Khá [7, 7.9], Trung bình [5.0, 6.9], Yếu [0,4.9].
  - Kết quả xuất dạng tham số output, không ghi xuống CSDL.

Tài liệu tham khảo

- Raghu Ramarkrishnan, Johannes Gehrke. *Database Management System*. 3rd Edition, Mc Graw Hill, 2003.
- Tutorials Point, Database Management System, www.tutorialspoint.com, 2015.
- Hector Garcia-Molina, Jeffrey D. Ullman and Jennifer Widom. *Database Systems, the Complete Book*, 2nd Edition, Prentice Hall, 2009
- Ramez Elmasri, Shamkant B. Navathe. *Fundamentals of Database Systems*. *7th Edition*, Pearson, 2015
- Silberschatz,Korth and Sudarshan, Database System Concepts (6th Edition), McGraw-Hill, 2010…….

Hết!
Cảm ơn Anh/ Chị đã theo dõi!

GVHD: TS. Phan Thị Thể

FIT – HCM-UTE

Email: thept@hcmute.edu.vn